TECHNICAL DETAIL

Title	Stored Cross-Site Scripting (XSS) via Malicious HTML Injection in Knowledge Detail Page
Reporter	Daffa Rahadya Atmawiguna (rahadya.daffaa@gmail.com)
Date	2025-12-28
Affected Asset	https://bertani.my.id/knowledge/detail
Severity	Medium
CVSS Score	5.4 CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:L/I:L/A:N
Proof of Concepts	Please see Steps to Reproduce section

SUMMARY

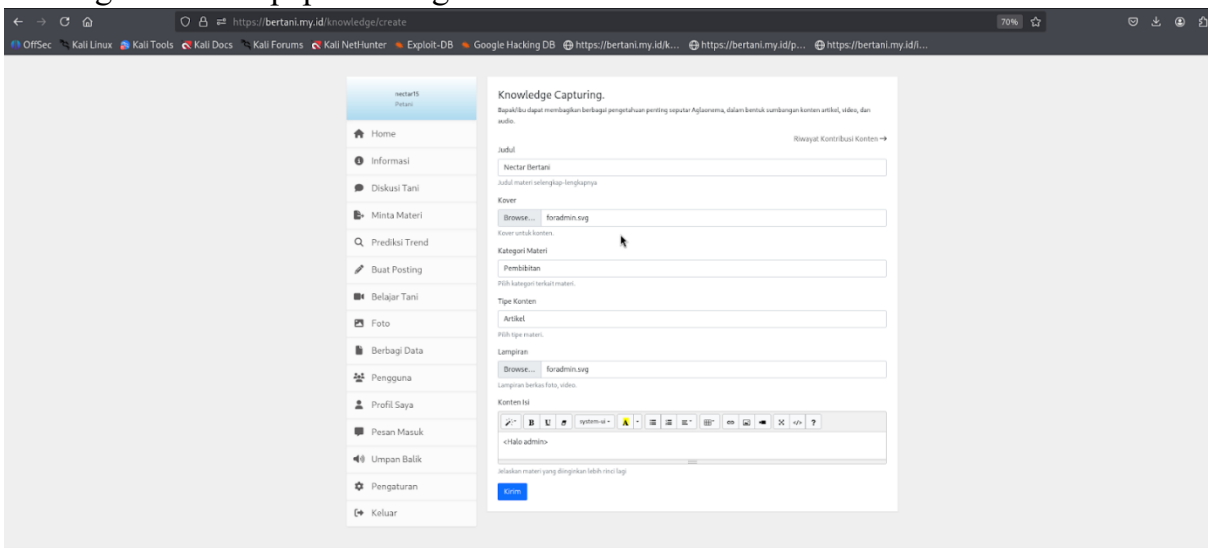
Kerentanan Stored Cross-Site Scripting (XSS) ditemukan pada url di bertani.my.id. Celah ini memungkinkan pengguna yang memiliki akses untuk membuat konten (seperti artikel) menyuntikkan skrip JavaScript berbahaya melalui parameter content. Karena input tersebut disimpan secara permanen di basis data tanpa sanitasi yang memadai, skrip tersebut akan dieksekusi secara otomatis di browser setiap pengguna yang mengakses halaman detail artikel tersebut di URL bertani.my.id/knowledge/detail

STEPS TO REPRODUCE

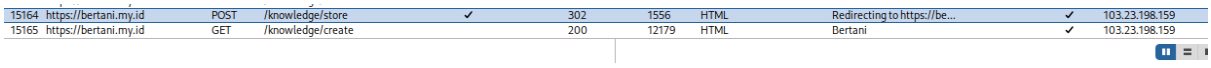
1. Login ke Akun Anda di URL https://bertani.my.id/login

2. Nyalakan Proxy sesuaikan dengan port BurpSuit
3. Aktifkan fitur Intercept pada Burp Suite .

4. Masuk ke url bertani.my.id/knowledge/create dan pilih tipe konten artikel dan isi dengan tulisan apapun di bagian konten isi lalu kirim



5. Lalu masuk ke burp suite dan kirim ke repeater yang post /knowledge/store



6. Setelah itu pada bagian name=”content” ubah menjadi img src=x onerror="alert"('Nectar')">

Dashboard | Target | Proxy | Intruder | Repeater | Collaborator | Sequencer | Decoder

1 2 3 4 5 6 7 8 9 10 11 12 13

Send | Cancel | Burp AI

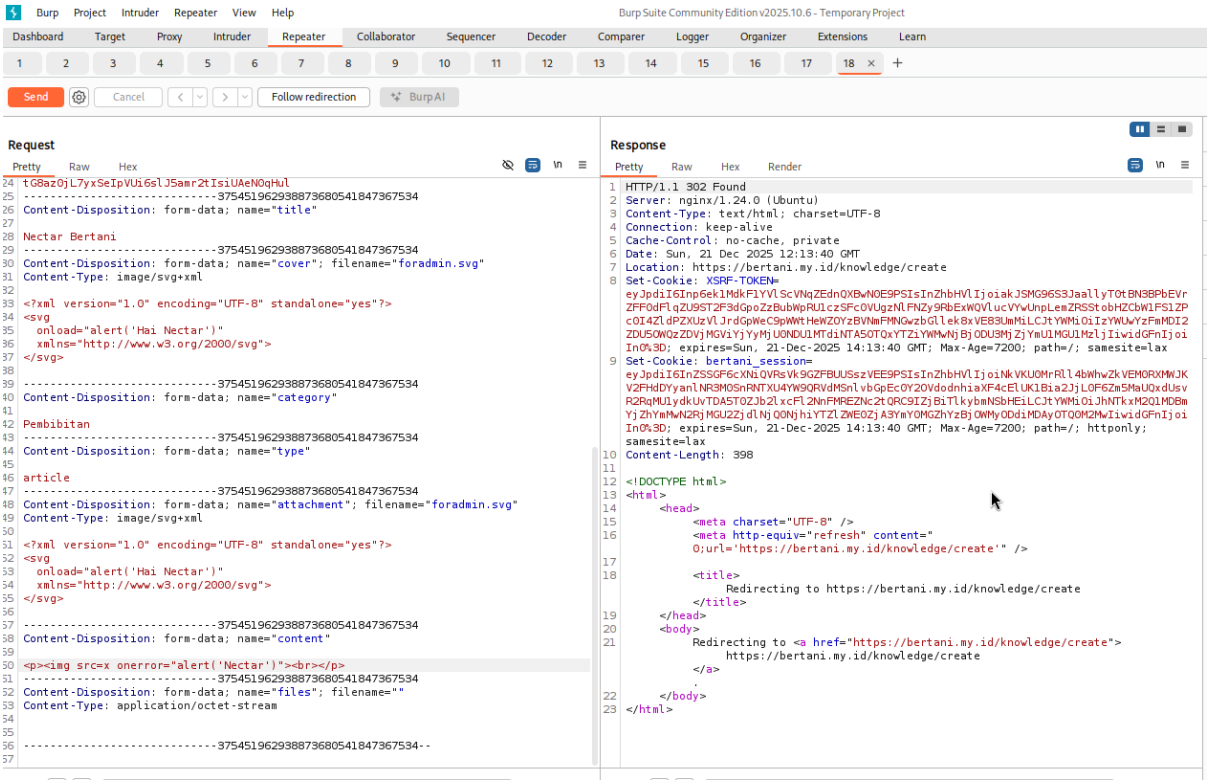
Request

Pretty | Raw | Hex

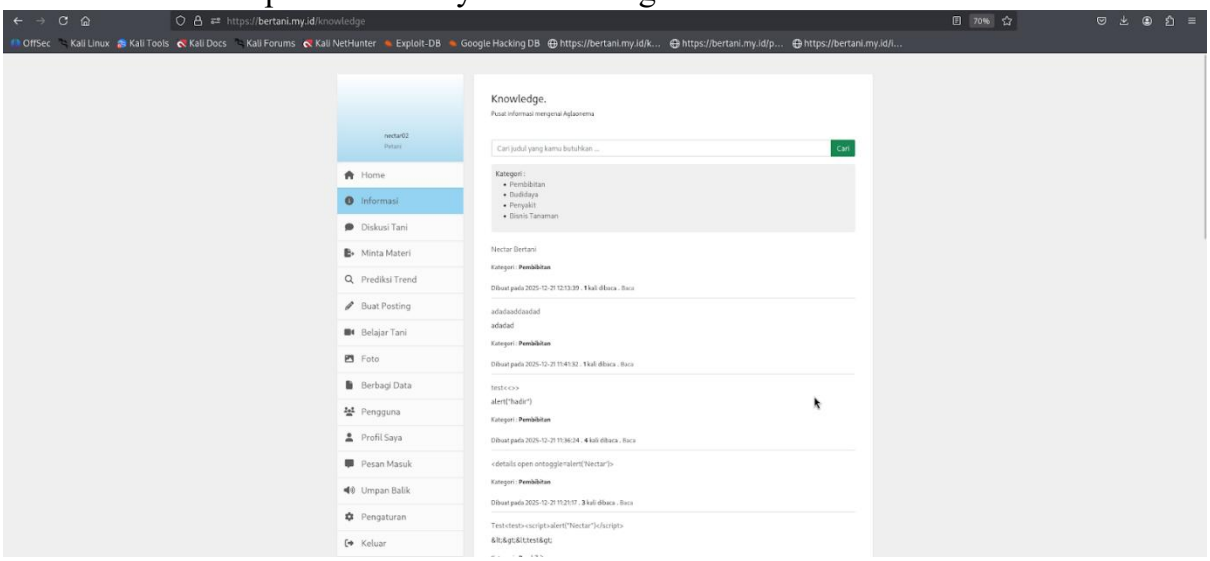
```
24 tG8az0jL7yxSeIpVUi6slJ5amr2tIsiUAeN0qHul
25 ----------------------------37545196293887368054184736753 4
26 Content-Disposition: form-data; name="title"
27
28 Nectar Bertani
29 ----------------------------37545196293887368054184736753 4
30 Content-Disposition: form-data; name="cover"; filename="foradmin.svg"
31 Content-Type: image/svg+xml
32
33 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
34 <svg
35   onload="alert('Hai Nectar')"
36   xmlns="http://www.w3.org/2000/svg">
37 </svg>
38
39 ----------------------------37545196293887368054184736753 4
40 Content-Disposition: form-data; name="category"
41
42 Pembibitan
43 ----------------------------37545196293887368054184736753 4
44 Content-Disposition: form-data; name="type"
45
46 article
47 ----------------------------37545196293887368054184736753 4
48 Content-Disposition: form-data; name="attachment"; filename="foradmin.svg"
49 Content-Type: image/svg+xml
50
51 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
52 <svg
53   onload="alert('Hai Nectar')"
54   xmlns="http://www.w3.org/2000/svg">
55 </svg>
56
57 ----------------------------37545196293887368054184736753 4
58 Content-Disposition: form-data; name="content"
59
60 <p>&lt;Halo admin&gt;<br></p>
61 ----------------------------37545196293887368054184736753 4
62 Content-Disposition: form-data; name="files"; filename=""
63 Content-Type: application/octet-stream
64
65
66 ----------------------------37545196293887368054184736753 4--
67
```

Search | 0 highlights

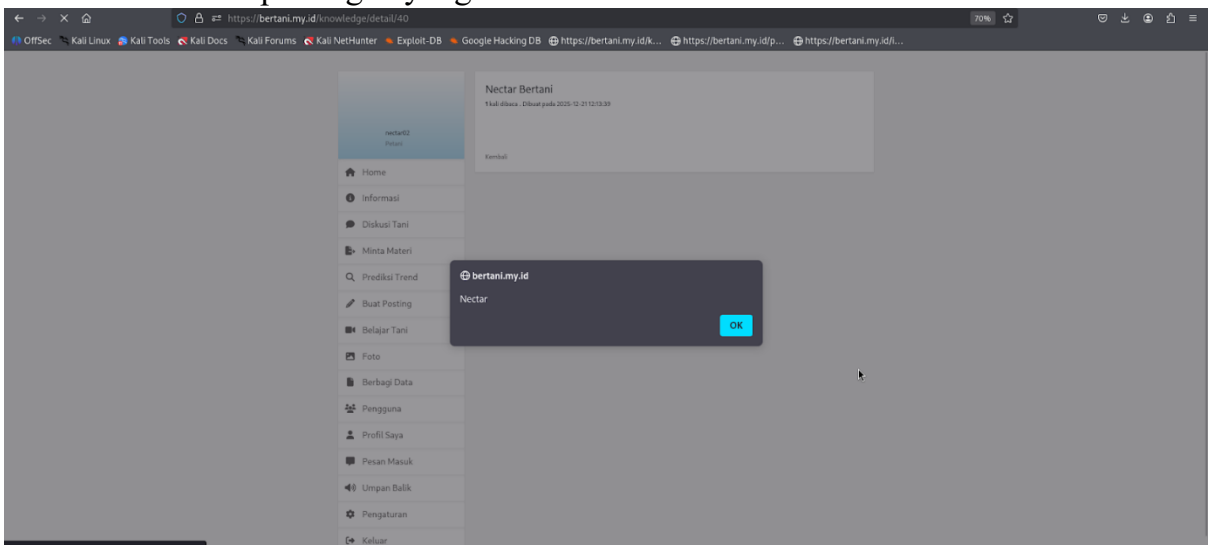
7. Seperti gambar dibawah



8. Maka saat admin sudah approve postingan kita akan muncul postingan yang kita buat tadi di https://bertani.my.id/knowledge.



9. Lalu Ketika tekan postingan yang sudah terima admin tadi akan muncul alert



EXPECTED RESULT

Aplikasi seharusnya melakukan pembersihan (sanitasi) terhadap semua input HTML dari pengguna atau melakukan *encoding* pada karakter-karakter khusus sebelum data ditampilkan di halaman detail. Tag seperti <img> dengan atribut *event handler* seperti onerror seharusnya tidak diperbolehkan atau dirender sebagai teks biasa sehingga tidak dapat mengeksekusi perintah JavaScript di sisi browser.

ACTUAL RESULT

Aplikasi gagal menyaring input berbahaya pada bagian "Konten Isi". Saat payload <img src=x onerror="alert('Nectar')">
 dikirimkan dan kemudian dilihat melalui halaman detail artikel, browser mengeksekusi atribut onerror tersebut karena sumber gambar (src=x) tidak valid. Hal ini dibuktikan dengan munculnya jendela peringatan (*alert box*) bertuliskan "Nectar" saat halaman diakses.

IMPACT

1. Defacement Masal: Penyerang dapat merusak reputasi situs dengan mengubah tampilan visual halaman detail artikel yang diakses oleh publik.
2. Penyebaran Malware: Skrip dimanfaatkan untuk mengarahkan pengunjung secara otomatis ke situs web berbahaya yang mengunduh perangkat lunak jahat (*malware*).

3. Pencurian Sesi Pengguna Umum: Tidak hanya admin, semua pengguna terdaftar yang membaca artikel tersebut berisiko kehilangan akun mereka.

RECOMMENDATION

1. Implementasi Sanitasi HTML: Gunakan pustaka sanitasi HTML yang terpercaya (seperti *HTML Purifier*) pada sisi server untuk menghapus tag dan atribut berbahaya (seperti script, onerror, onload) sebelum data disimpan ke database.
2. Output Encoding: Pastikan data yang diambil dari database selalu di-*encode* sebelum ditampilkan ke browser pengguna untuk memastikan tag HTML tidak dirender sebagai kode aktif.
3. Content Security Policy (CSP): Terapkan kebijakan CSP yang ketat untuk melarang eksekusi skrip *inline* dan membatasi sumber skrip eksternal yang diizinkan.
4. Gunakan Atribut HttpOnly: Pastikan semua cookie sensitif (terutama session ID) menggunakan atribut HttpOnly untuk mencegah akses melalui JavaScript.

REFERENCES

1. **PortSwigger - Stored XSS:** https://portswigger.net/web-security/cross-site-scripting/stored
2. **OWASP - Cross Site Scripting (XSS):** https://owasp.org/www-community/attacks/xss/